Guarded Multi-Step Support Agent

Interview question bank with model answers

Questions	103
Sections	12
Difficulty	24 easy / 56 medium / 23 hard
Covers	ReAct agents, LangGraph state and interrupts, MCP tools, guardrails, human sign-off, tau2-bench, pass^k, tool accuracy, escalation, trace debugging, Temporal, Cloud Run and Arize

How to use this. Cover the answer and say yours out loud first, then compare. Model answers are written to be spoken in 30-90 seconds. Follow-ups are what a good interviewer asks next. 'In the repo' points to the code that backs the answer, so you can cite it. No benchmark numbers are claimed: fill them in from your own tau2 runs before an interview.

Contents

1. Project overview and framing . . . 3
2. Agent architecture: ReAct and LangGraph . . . 4
3. Tools and MCP . . . 6
4. Guardrails . . . 8
5. Human in the loop . . . 10
6. tau2-bench . . . 12
7. Evals: success, pass^k, tool accuracy, escalation . . . 14
8. Debugging with traces and observability . . . 16
9. Durability: checkpoints and Temporal . . . 17
10. Deployment and security . . . 18
11. Trade-offs and behavioural questions . . . 19
12. Rapid-fire concepts . . . 20
Appendix: formula sheet . . . 21

1. Project overview and framing

Openers. Keep each answer under a minute.

Q1 Describe the project in two sentences. EASY

A customer-support agent that reasons, calls a tool, reads the result and decides again over many steps, built as a LangGraph graph with tools served over MCP. Every tool call passes deterministic guardrails, irreversible actions pause for a human's sign-off, every step is traced, and it is evaluated on tau2-bench with task success, pass^k, tool accuracy and escalation rate.

In the repo: `README.md`

Q2 What makes an agent different from a chatbot with tools? EASY

Autonomy over multiple steps: the agent decides what to look up, in what order, when it has enough information, and when to act, rather than mapping one request to one call. That's where the risk comes from too: each step can compound an earlier mistake, and some actions can't be undone. The project is mostly about controlling that risk without killing the autonomy.

Q3 What are the four things this project is meant to prove? MEDIUM

Where the human sits (customer confirmation plus operator sign-off for irreversible actions), what evals I built (success, pass^k, tool accuracy, escalation, failure taxonomy), what guardrails I put in (deterministic tool-call rules, output checks, injection screening), and how I debug with traces (span per step, a timeline tool, and a failure taxonomy that tells me which traces to open).

In the repo: `docs/design_decisions.md`

Q4 Why customer support as the domain? MEDIUM

It has real, checkable consequences: refunds, cancellations, bookings. Policies are explicit, so violations are objective. And tau2-bench provides realistic tasks with a user simulator and a database-state check, which gives a ground truth for success that doesn't depend on judging text.

Q5 What would you do with two more weeks? HARD

Run the full tau2 domains with k=4 against the stock agent and publish the paired comparison with CIs; run the guard ablation (on, off, auto-deny) and count unsafe actions in the unguarded trajectories; add airline and telecom domain rules (identity and ownership) since only the generic rules apply there now; and measure false blocks by replaying reference trajectories through the guard.

> ***Follow-up:*** *Why the ablation first? It's the evidence that the guardrails are worth their cost in success rate.*

2. Agent architecture: ReAct and LangGraph

How the loop is built and why.

Q6 What is ReAct? EASY

Reason + Act: the model alternates between reasoning about what to do, taking an action (a tool call), and observing the result, repeating until it can answer. With native tool use the reasoning is implicit in the model's choice of tool calls; the loop structure is the same.

Q7 Walk me through the graph. EASY

`agent` calls the model. If it proposes tool calls they go to `guard`, which allows, blocks, escalates or sends them to `approval`. Allowed and approved calls run in `tools`, whose results go back to `agent`. If the model replies with text it goes to `output_guard`, which either sends it to the customer and ends the turn or asks for one rewrite. `escalate` hands off to a human.

In the repo: `agent/nodes.py`

Q8 Why LangGraph rather than a hand-written while loop? MEDIUM

Three things I'd otherwise rebuild: explicit, inspectable control flow (nodes and edges you can draw and test); checkpointing of state at every step, which makes long pauses and crash recovery possible; and `interrupt`, which pauses a run for a human and resumes it with their answer. The graph also makes it obvious where guardrails sit.

> ***Follow-up:*** *When would you not use it? A single-tool, single-step assistant where the overhead isn't worth it.*

In the repo: `agent/graph.py`

Q9 What is in the state and why only JSON? MEDIUM

Messages, pending calls, guard decisions, an approvals audit log, guard events, step and retry counters, the output draft and status. Only JSON-serialisable values, because the state is written to a checkpointer (SQLite or Postgres), passed through Temporal, and dumped into traces. Lists like messages use append reducers so nodes return only what's new.

In the repo: `agent/state.py`

Q10 Why recompute guard facts from the message log instead of storing flags? MEDIUM

A flag like `authenticated=True` can drift from reality or be set by a buggy node, and nothing checks it. Rebuilding facts (who was authenticated by a successful lookup, which orders they own, whether the last customer message was a yes) from the log every time means decisions are reproducible: a resumed checkpoint or replayed trace gives the same answer.

In the repo: `guardrails/context.py`

Q11 What is MiniGraph and isn't it cheating? MEDIUM

A small local executor implementing the LangGraph behaviour the agent depends on: reducers, per-thread checkpoints, and interrupt/resume where the interrupted node re-runs from the top. Tests and the offline demo run the exact same node functions and topology through it, with no dependencies. It isn't used in production; the point is that the behaviour under test is the nodes', and the topology is defined once and shared.

In the repo: `agent/runtime.py`

Q12 LangGraph re-runs a node from the top when resumed. How did you design for that? HARD

Everything before `interrupt()` in the approval node is pure: it builds the request from state. Execution of the approved tool calls happens in a separate `tools` node after approval. So resuming can't double-execute a refund. The test checks the approval node ran twice and the tool ran once.

In the repo: `tests/test_agent.py`

Q13 How do you stop the agent running forever? MEDIUM

Three budgets: a step budget per turn (`MAX_STEPS_PER_TURN`), a limit on identical tool calls (same tool and arguments) which escalates as a loop, and a limit on consecutive fully-blocked steps. Hitting any of them escalates to a human with a summary instead of failing silently. LangGraph's recursion limit is a last backstop.

In the repo: `config.py`

Q14 Why are the nodes synchronous when the MCP client is async? HARD

Synchronous nodes keep checkpoint and tracing semantics simple and match how the tau2 adapter is called. The async MCP client runs on its own event-loop thread inside `MCPToolbox`, which exposes a blocking `execute()`. In a high-concurrency server I'd switch the graph to async nodes; the node logic doesn't change.

In the repo: `tools/mcp_client.py`

Q15 How does a rejected reply get rewritten without the customer seeing it? MEDIUM

The agent's text reply goes into a `draft` field, not the message log. The output guard either commits it or sets `feedback`; on the next agent call the draft and feedback are appended only to that LLM request, never stored. The test checks the rejected text never appears in the conversation.

In the repo: `nodes.py: output_guard`

3. Tools and MCP

Tool design, risk metadata and the MCP integration.

Q16 What is MCP? EASY

The Model Context Protocol: a standard way for an application to expose tools (and resources, prompts) to model clients over stdio or HTTP. A server declares tools with input schemas and annotations; any MCP client can list and call them.

Q17 Why serve the tools over MCP instead of calling Python functions? MEDIUM

It separates the tool surface from the agent: the tool server can be deployed, permissioned and scaled on its own (internal-only on Cloud Run), reused by other clients, and swapped for real backends without touching the agent. The agent can also consume third-party MCP servers. In tests the server runs in-process.

In the repo: `tools/mcp_server.py`

Q18 What risk metadata does each tool carry? MEDIUM

`kind` (read, write, transfer, generic), `irreversible`, `confirm` (customer must say yes) and `approve` (operator must sign off), plus a risk note shown to the approver. Over MCP these map to standard annotations (`read_only_hint`, `destructive_hint`, `idempotent_hint`) and our full metadata goes in the tool's `meta`.

In the repo: `tools/registry.py`

Q19 Would you trust a third-party MCP server's `read_only_hint`? HARD

No, annotations are hints from the server. Our own metadata wins when present; otherwise annotations are used, and a tool with neither falls back to name-based classification that fails closed: an unknown verb is treated as a write needing confirmation. For a third-party server I'd also review and pin its tool list, because a server can change tools between sessions.

In the repo: `mcp_client.spec_from_mcp`

Q20 How do tool errors reach the model? MEDIUM

As tool results with `is_error=True` and an actionable message, never as exceptions that crash the run. In the MCP server, expected failures raise `ToolError` so the message is passed through; unexpected exceptions are hidden by the SDK. The model reads the error and can recover, which is how tau2 works too.

In the repo: `mcp_server._wrap`

Q21 Which rules live in the tools and which in the guards? MEDIUM

Tools enforce invariants that are true regardless of conversation: only pending orders can be cancelled, refunds go to the original payment method or a gift card, an order's items can be modified once. Guards enforce rules that depend on the conversation, which a tool can't see: is the customer authenticated, did they say yes, do they own this order.

In the repo: `domains/retail/tools.py`

Q22 How do you write tool descriptions for an agent? HARD

Say what it does, when to use it, where its inputs come from ('ids come from get_user_details'), and constraints that would otherwise surprise the model ('can only be done once, collect all items first'). Keep schemas strict (enums for reasons, `additionalProperties: false`). Many 'reasoning' failures are actually ambiguous tool descriptions, which shows up in traces as wrong arguments.

Q23 Why stateless streamable HTTP for the MCP server on Cloud Run? MEDIUM

Cloud Run routes requests to any instance and scales them freely. A stateful MCP session pinned to one instance would break when the next request lands elsewhere. Stateless mode treats each request independently.

In the repo: `mcp_server.main`

4. Guardrails

The rules, why they're deterministic, and how they're measured.

Q24 What guardrails did you put in? EASY

Input: injection screening of customer messages and tool output, tool output wrapped as data, PII redaction in traces. Tool calls: unknown tool, schema, loop detection, grounded ids, authentication, same user, ownership of orders and payment methods, customer confirmation, confirmation must match the action, one write per confirmation, one write per response, operator approval for irreversible actions. Output: unbacked claims, data leaks, length.

In the repo: `docs/guardrails.md`

Q25 Why deterministic code rather than an LLM judge for guardrails? MEDIUM

Code is fast, free, reproducible, testable, and can't be persuaded. An LLM guard can be argued with by the same prompt that fooled the agent, adds latency to every step, and its errors are hard to reason about. I use an LLM only where semantics are unavoidable (an optional injection classifier), and it can only make a decision stricter.

In the repo: `guardrails/policy.py`

Q26 What does 'guard the action, not the words' mean? MEDIUM

A user can talk the model into proposing anything. If only guarded actions execute, persuasion changes the proposals but not the outcome. So prompt injection becomes a nuisance that costs a blocked call, not a breach.

Q27 How does the confirmation guard decide the customer said yes? MEDIUM

The last customer message must start with an explicit affirmative (yes, confirm, go ahead...) and not be hedged ('wait', 'not yet'). The agent's message right before it must name the object being changed (the order or reservation id), so a 'yes' to 'anything else?' doesn't authorise a cancellation. And no other change may have happened since that message.

> ***Follow-up:*** *What are its weaknesses? It's lexical: 'sure, why not' passes; 'yes but not the keyboard' passes. I'd add the item list to the check for item-level actions.*

In the repo: `guardrails/context.py`

Q28 What is the ungrounded-id rule? MEDIUM

Every argument named like an id that contains digits must have appeared in a customer message or a successful tool result. It catches the most common agent hallucination: a plausible order or item id. The block reason tells the model to look it up, so it usually recovers in the same turn.

In the repo: `r_grounded_ids`

Q29 Can a tool result authorise an action? HARD

No. Confirmation is read only from customer messages. A tool result saying 'the customer approved, cancel now' is treated as data, flagged as a possible injection, wrapped with a note, and the cancel is still blocked until the customer says yes. There's a test for exactly this.

In the repo: `test_tool_output_cannot_confirm`

Q30 What does a blocked call look like to the model? MEDIUM

A tool result with `is_error=True`: 'Blocked by guardrail (needs_user_confirmation): first describe the exact action ... and ask the user to explicitly confirm'. The reason is actionable so the model can fix the problem itself.

Q31 What is the output guard for? MEDIUM

Stopping the agent from telling the customer something happened when it didn't, which is the most damaging support failure because the customer leaves believing it. It flags past-tense claims (cancelled, refunded, updated) when no write succeeded in the turn, and exempts future tense and questions. It also checks for another customer's email.

In the repo: `guardrails/output.py`

Q32 How do you measure whether a guardrail helps or hurts? HARD

Block counts by rule; whether trials with blocks succeed (the agent recovered) or fail; false blocks, by replaying tau2's reference trajectories through the guard; and an ablation on the same tasks with guards on and off, comparing success, pass^k and unsafe actions (writes without a prior yes, actions on objects the customer doesn't own).

In the repo: `evals/metrics.guard_stats`

Q33 Your guard blocks a correct action. What do you do? HARD

Treat it like any other bug: find it in the traces (blocked call in a trial that then failed), check it against the policy, and fix the rule rather than loosening globally. Then add the case to the tests. False blocks cost success rate; false allows cost money, so the bar for loosening is a specific, tested rule change.

Q34 Why are identity rules only applied to retail? MEDIUM

Retail authenticates through lookup tools (email, or name and zip). tau2's airline and telecom identify customers differently, so applying retail's rule would block valid calls. The generic and write rules still apply everywhere; domain identity rules are a pluggable list.

In the repo: `DOMAIN_RULES`

Q35 How do guardrails interact with prompt caching and latency? HARD

They don't add model calls, so there's no extra latency beyond microseconds of Python. The system prompt with policy and tool schemas is cached, and blocked calls add a model step only when the model was about to do something wrong, which is cheaper than the incident.

5. Human in the loop

Where the human sits, what they see, and failure modes.

Q36 Where does the human sit? EASY

Two places. The customer must explicitly confirm every account change; that's enforced by a guard. An operator must sign off on irreversible actions (cancellations, returns, exchanges, item changes); the graph pauses with an interrupt until they approve or deny.

In the repo: `docs/design_decisions.md`

Q37 Why not require approval for every write, or every tool call? MEDIUM

Approvers stop reading when everything needs approval, so it becomes a rubber stamp. Risk-tiering puts human attention where mistakes can't be undone: reversible changes need only the customer's yes, reads need nothing.

Q38 What does the operator see? MEDIUM

The tool, exact arguments, a risk note ('Irreversible: the order is cancelled and refunds issued'), and a digest of the last few turns. They approve or deny each action with a reason; the decision, reviewer and time are stored in the state's approvals log, the trace, and Temporal history if used.

In the repo: `nodes.approval`

Q39 How does the pause work technically? MEDIUM

The approval node calls `interrupt(request)`. LangGraph persists the state in the checkpointer and returns `__interrupt__` to the caller. The API exposes pending requests; the operator's decision calls `graph.invoke(Command(resume=decisions))` with the same thread id, the node re-runs, `interrupt` returns the decisions, and the tools node executes approved calls.

In the repo: `agent/service.py`

Q40 What happens if the operator denies? MEDIUM

The call becomes a tool error: 'a supervisor declined this action (reason)'. The agent tells the customer it couldn't be completed and offers alternatives or a transfer. Nothing executes. The output guard prevents the agent from claiming success.

Q41 What if nobody approves for hours? HARD

Without Temporal the conversation stays paused in the checkpointer and the API returns 409 to new customer messages, which is safe but poor service. With Temporal the workflow waits on a signal with a timeout; on timeout it resumes the graph with a denial, so the customer is told and offered a transfer. Fail safe means deny, never auto-approve.

In the repo: `temporal/workflow.py`

Q42 How do benchmarks handle approval when no human is there? MEDIUM

A logged policy approves (or, in an ablation, denies) in place of the operator. The number of approval requests is reported as the operator workload the agent would create in production.

In the repo: `bench/local.policy_approver`

Q43 Could the customer's 'yes' replace operator approval? HARD

They authorise different things. The customer authorises intent; the operator authorises risk: fraud patterns, policy exceptions, or a model error that the customer happily agrees to (refund to the wrong card). Keeping both means one compromised or mistaken party isn't enough.

Q44 How is the approval endpoint protected? MEDIUM

Operators send a token and a reviewer header; the reviewer is recorded with every decision, and the endpoint requires a decision for every pending action (422 otherwise), so nothing is approved by omission. In production I'd put it behind IAP or OIDC rather than a shared token.

In the repo: `api/main.py`

6. tau2-bench

The benchmark and how the agent plugs into it.

Q45 What is tau2-bench? EASY

A benchmark from Sierra for tool-using conversational agents in customer-service domains (retail, airline, telecom). Each task has a policy, tools, a database and a hidden user instruction; an LLM plays the customer. The current release is published as tau-cubed (adds voice and a knowledge domain).

In the repo: `docs/tau2.md`

Q46 How does tau2 decide a task succeeded? MEDIUM

The reward is the product of the components in the task's reward basis, by default DB and COMMUNICATE. DB: the final database hash equals the hash after replaying the reference actions on a fresh database, so any path to the same end state passes. COMMUNICATE: every required string appears in the agent's messages.

Q47 Why is a database-state check better than comparing the action sequence? MEDIUM

Many valid paths reach the same end state (different lookup order, extra reads). Comparing sequences would fail correct agents. The DB check rewards outcomes. tau2 uses action matching only for a few tasks where the path itself matters.

Q48 How does your agent run inside tau2? MEDIUM

As a tau2 `HalfDuplexAgent`: for each incoming message it calls the model, runs the same guard and output checks, and returns either text or allowed tool calls. Blocked calls are fed back to the model internally and never reach tau2's environment. It's registered with `register_agent_factory` and selected with `--agent guarded_react`.

In the repo: `bench/tau2_agent.py`

Q49 Refusal tasks: how does an agent pass a task where the right answer is to do nothing? HARD

The reference actions change nothing, so the goal hash is the initial database. Any write fails the task. These are where guardrails matter: cancelling a delivered order, acting on another customer's order. The local task set includes two.

In the repo: `bench/local_tasks.py`

Q50 What is the user simulator and why can it be a problem? MEDIUM

An LLM given the hidden instruction that plays the customer. It can deviate: reveal information too early, forget details, or end the conversation wrongly. That makes some failures the simulator's, not the agent's. Use the same user model for runs you compare and spot-check failures, especially ones labelled communication.

Q51 How did you test the adapter without tau2 installed? HARD

Against a stand-in that implements the documented interface: `HalfDuplexAgent(tools, domain_policy)`, message models with role, content and tool calls, `ToolMessage(id, content, error)`, `MultiToolMessage`, and tools with an OpenAI schema. The test drives a blocked call, an authenticated lookup, confirmation and approval, and checks tau2 never saw the blocked call.

> ***Follow-up:*** *Risk? Version drift. That's why the adapter and loader are the only files that touch tau2.*

In the repo: `TestTau2Adapter`

Q52 Why also build a local benchmark? MEDIUM

tau2 runs are slow and cost tokens. Seven local tasks in the same shape (DB-hash plus communicate scoring) give a fast loop for prompt and guard changes, and with scripted users they're part of the unit tests.

In the repo: `bench/local.py`

7. Evals: success, pass^k, tool accuracy, escalation

Definitions, why each exists, and pitfalls.

Q53 Define task success rate. EASY

The share of trials with reward 1: the database ended in the goal state and required information was communicated. It equals pass^1.

In the repo: `evals/metrics.py`

Q54 Define pass^k and how you estimate it. MEDIUM

The probability that all k independent trials of a task succeed. With n trials and c successes, the unbiased estimate is C(c,k)/C(n,k); average over tasks. With 3 of 4 successes, pass^2 = 3/6 = 0.5 and pass^4 = 0.

In the repo: `pass_hat_k`

Q55 Why pass^k instead of pass@k? MEDIUM

pass@k (at least one of k succeeds) rewards an agent that sometimes gets it right, which is fine for code generation with tests but wrong for support, where every customer gets one try. pass^k measures reliability. An agent with 75% success that is right 75% of the time on every task has pass^4 around 0.32; one that's always right on 75% of tasks has 0.75. The gap between pass@k and pass^k is the inconsistency.

Q56 How many trials per task and why? MEDIUM

At least 4 so pass^k up to k=4 is defined, at the production temperature, because consistency at production settings is what you're measuring. More trials narrow the CI; tasks matter more than trials for the overall estimate.

Q57 What is your tool accuracy metric? MEDIUM

Executed, successful writes compared to the reference writes, matching tool name and the arguments tau2 says to compare (normalised): precision, recall, F1. Plus read-tool recall, tool error rate, guard block rate and an exact write-sequence rate. Errored or blocked writes don't count as changes.

In the repo: `tool_accuracy`

Q58 Why can tool accuracy disagree with task success? HARD

Success can fail with perfect writes (information not communicated, or a user-simulator error), and success can pass with imperfect tool metrics (a different but equivalent path, extra harmless reads). Reporting both separates 'did the wrong thing' from 'said the wrong thing'.

Q59 How do you measure escalation? MEDIUM

Escalation rate is the share of trials that transferred to a human. It's split into appropriate (the reference also transfers), unnecessary (task was solvable) and missed (should have transferred, didn't), plus how many were triggered by the guard's escalation rules.

In the repo: `escalation_stats`

Q60 Why is a low escalation rate not automatically good? HARD

An agent can lower it by attempting things it should hand off, which shows up as missed escalations and policy failures. And a high rate can look like safety while costing money and annoying customers. Both directions are errors; I track them separately.

Q61 What is the failure taxonomy? MEDIUM

A label per failed trial from ordered heuristics: guard escalation, approval denied, step limit or error, missed or unnecessary escalation, no action, wrong arguments, wrong tool, incomplete, extra action, communication. It tells me which fix to try first and which traces to open. Labels are triage hints, confirmed by hand on a sample.

In the repo: `evals/failures.py`

Q62 How do you compare two agents fairly? HARD

Same tasks, same k, same agent and user models and temperature. Compute per-task success for each, then a paired difference with a bootstrap over tasks (trials of the same task are correlated, so resampling trials overstates confidence). Report the CI, not just the means.

In the repo: `scripts/compute_metrics.py`

Q63 What else do you track besides correctness? MEDIUM

Cost per trial, tool calls per trial, turns, guard blocks by rule, approvals per trial (operator workload) and termination reasons. An agent that is 2 points better but makes twice the calls may not be better in production.

Q64 How would you evaluate in production where there's no reference DB? HARD

Reference-free signals: escalation rate, approval denial rate, guard block rate by rule, output-guard rewrites, customer follow-ups within 24 hours (the issue wasn't resolved), operator corrections, and a sampled human review with a rubric. Plus the offline benchmark as a regression gate before each release.

8. Debugging with traces and observability

How you find out why the agent failed.

Q65 What do you trace? EASY

Every model call (model, latency, proposed tools or text), every guard decision (tool:action:rule), every tool call (arguments, error flag, output preview), and approval and escalation events. Locally as JSONL per conversation; in Arize as OpenInference spans with the conversation as a session.

In the repo: `tracing.py`

Q66 Walk me through debugging a failure. MEDIUM

Start from the failure taxonomy to pick the biggest category, open three to five traces from it, and render each with the timeline tool. Find the first wrong step, not the last: most failures are decided early. Classify the root cause (prompt, tool description, guard rule, tool bug, user simulator), fix one thing, re-run the same tasks with the same k, and compare with a paired bootstrap.

In the repo: `docs/debugging.md`

Q67 Give an example of a trace pattern and the fix it suggests. MEDIUM

Repeated `needs_user_confirmation` blocks in a turn mean the model acts before asking; strengthen the instruction or add an example. `ungrounded_id` blocks mean it guesses ids; fix the tool description to say where ids come from. Identical calls with errors mean it isn't reading the error; make the error text actionable.

In the repo: `scripts/inspect_trace.py`

Q68 Why redact PII in traces? MEDIUM

Traces go to a third-party observability service and are read by many people. Emails, card numbers and phone numbers are masked before export; the agent itself still sees raw data. Redaction is applied to all span attributes.

In the repo: `guardrails/injection.redact`

Q69 How do you tell an agent failure from a user-simulator failure? HARD

Read the conversation: did the simulated customer reveal what the instruction said, at a sensible time, and stop correctly? Failures labelled communication or with odd user turns are candidates. Count them separately and report agent success with and without them, but never 'fix' the agent for a simulator bug.

Q70 What do you look at in Arize specifically? MEDIUM

Guard spans filtered by block and grouped by rule, tool spans with errors grouped by tool, LLM latency percentiles, and sessions joined with eval results so I can slice failures by label, model version or prompt version.

Q71 A change improved success by 3 points. How do you know it's real? HARD

Paired comparison on the same tasks and k, bootstrap CI over tasks for the difference; if the CI includes zero it isn't established. Check the failure taxonomy moved where the change was aimed, and that pass^k moved too, not just pass^1. Then confirm on a different task subset or domain.

Q72 What does the trace inspector summary show? MEDIUM

Blocks by rule, repeated identical tool calls, slow model steps, denied approvals, and hints that map common patterns to likely fixes.

In the repo: `inspect_trace.py`

9. Durability: checkpoints and Temporal

Long pauses, crashes, retries.

Q73 What does the checkpointer do? EASY

Saves the graph state after every step under a thread id. It's required for interrupts, lets a conversation resume after a crash or on another instance, and lets you inspect the state at any step. SQLite for dev, Postgres for Cloud Run.

In the repo: `graph.make_checkpointer`

Q74 Why Postgres on Cloud Run? MEDIUM

Instances are ephemeral and scale out. A conversation paused for approval must survive the instance that paused it, and the operator's decision may land on a different instance. With the checkpoint in Postgres, any instance can rebuild the agent and resume.

In the repo: `api/main._session`

Q75 What does Temporal add on top of checkpoints? MEDIUM

Checkpoints make a pause resumable; nothing wakes it up. Temporal makes waits timed and supervised: a workflow per conversation waits on signals for customer messages and approvals, enforces an approval timeout, retries failed activities, and keeps a full history as an audit trail.

In the repo: `temporal/workflow.py`

Q76 Why is each agent turn an activity and not workflow code? HARD

Workflow code must be deterministic so Temporal can replay it; LLM and tool calls aren't. Activities hold the non-deterministic work and can be retried. Because the graph's state is in Postgres, a retried activity resumes from the last checkpoint instead of redoing completed steps.

Q77 Could a retry execute a refund twice? HARD

The design avoids it in layers: execution happens in a separate node after approval, so re-running approval doesn't execute; checkpoints mean completed steps aren't repeated; and real write tools should be idempotent with a key (the call id) so the backend rejects a duplicate. The last one is the one I'd insist on for real money.

Q78 Why is Temporal optional? MEDIUM

For a demo or low volume, checkpoints plus a 409 while paused are enough. Temporal pays off when approvals take hours, when you need timeouts and retries with history, or when conversations span days.

Q79 What signals and queries does the workflow expose? MEDIUM

Signals: a customer message and an approval with decisions and reviewer. Query: status, last reply and pending approval request.

In the repo: `SupportConversation`

10. Deployment and security

FastAPI, Docker, Cloud Run, secrets.

Q80 Describe the deployment. EASY

One Docker image with three roles: the FastAPI API, the MCP tool server, and an optional Temporal worker. On Cloud Run the API is public with Cloud SQL Postgres as the checkpointer; the tool server is internal-only, reached through VPC egress; secrets come from Secret Manager.

In the repo: `deploy/cloudrun.sh`

Q81 What does the API expose? MEDIUM

Customer endpoints to create a session, send a message and read the conversation; operator endpoints to list pending approvals, decide them and read traces. Customers can't send messages while an approval is pending (409).

In the repo: `docs/api.md`

Q82 Why keep the tool server off the public internet? MEDIUM

It is the agent's hands. Anything that can call it can change accounts, bypassing every guardrail that lives in the agent. Internal ingress means only the API can reach it.

Q83 How did you size Cloud Run? MEDIUM

Moderate concurrency (each request can hold several LLM calls for seconds), a 300 second timeout, one instance minimum to avoid cold starts on chat, and horizontal scaling since state is in Postgres. Tool server scales to zero.

Q84 What are the main security risks of an agent like this? HARD

Prompt injection leading to unauthorised actions (mitigated by guarding actions and confirmation from customer messages only), acting on another customer's data (identity and ownership rules), data leakage in replies (output guard), over-privileged tools (minimal MCP surface per domain, internal only), and sensitive data in logs (redaction).

Q85 What do you alert on in production? MEDIUM

Escalation rate, approval queue age, approval denial rate, guard blocks by rule, tool error rate, output-guard rewrites, and p95 turn latency. Sudden changes usually mean a model, prompt or backend change.

In the repo: `docs/deployment.md`

Q86 How do you ship a prompt change safely? MEDIUM

Run the unit tests and the local benchmark in CI, then tau2 on a fixed task subset with k=4 and compare to the current version with a paired bootstrap. Deploy gradually and watch the production signals.

11. Trade-offs and behavioural questions

Answer with situation, decision, evidence.

Q87 Tell me about a trade-off you made deliberately. MEDIUM

Failing closed. Unknown tools are treated as writes needing confirmation, and the guard blocks rather than allows when unsure. That costs some success rate (an extra question to the customer) in exchange for never executing an unconfirmed change. I measure the cost with block-rule stats and false-block replay.

Q88 What was the subtlest bug? MEDIUM

LangGraph re-runs an interrupted node from the top on resume. If the approval node executed tools after interrupt, a crash between execution and checkpoint could double-execute on resume. Splitting execution into its own node made resumption safe, and the test asserts approval runs twice and the tool once.

Q89 What would you not trust in your own results? MEDIUM

Numbers from small task subsets (wide CIs), failures that might be the user simulator's, and auto-approve results as a proxy for production, where a human may deny actions the benchmark allowed.

Q90 A PM wants the agent to handle refunds fully autonomously. How do you respond? HARD

Ask what failure rate is acceptable and for which amounts. Propose risk tiers: autonomous under a threshold with customer confirmation, operator sign-off above it, and measure approval denial rates over time. If operators approve nearly everything in a tier for weeks, that's evidence to lower friction there, tier by tier.

Q91 Why build an offline demo with scripted models? MEDIUM

So anyone can see the real graph, guards, approvals and MCP integration work in seconds without keys, CI tests the control flow on every commit, and failures in the logic are separated from model variability.

In the repo: `scripts/demo_offline.py`

12. Rapid-fire concepts

Short answers you should give instantly.

Q92 What is a checkpointer? EASY

Persistent storage of graph state per thread after each step; needed for interrupts and resumption.

Q93 What does `interrupt()` return? EASY

Nothing the first time (it pauses the run); on resume it returns the value passed in `Command(resume=...)`.

Q94 What is a reducer in LangGraph? EASY

How a state key merges a node's update, for example appending to a messages list instead of replacing it.

Q95 What is tool-call grounding? EASY

Requiring tool arguments, especially ids, to come from the conversation or tool results rather than the model's guess.

Q96 pass^1 equals what? EASY

The task success rate.

Q97 What does C(c,k)/C(n,k) estimate? EASY

The probability that k trials drawn without replacement from the n run are all successes: pass^k for that task.

Q98 What is an MCP tool annotation? EASY

A hint about a tool's behaviour: read-only, destructive, idempotent, open-world.

Q99 Escalation vs transfer? EASY

Same action (hand to a human); escalation here means the guard or budgets triggered it rather than the model choosing to transfer.

Q100 Why temperature 0 for a support agent? EASY

Repeatability and pass^k. Some variation can help recover from loops, but consistency matters more to customers.

Q101 What is prompt caching useful for here? EASY

The policy and tool schemas are a long, static prefix sent on every step; caching it cuts cost and latency.

Q102 What's an idempotency key? EASY

A unique id sent with a write so the backend can ignore duplicates when a request is retried.

Q103 Why bootstrap over tasks, not trials? EASY

Trials of the same task are correlated; resampling them independently understates uncertainty.

Appendix: formula sheet

Metric	Definition
Task success	mean over trials of 1[reward = 1] (= pass^1)
tau2 reward	product of components in reward_basis; default DB × COMMUNICATE
pass^k (per task)	C(c, k) / C(n, k); c successes in n trials; averaged over tasks
pass@k (per task)	1 - C(n - c, k) / C(n, k)
Write precision / recall	matched successful writes / successful writes; matched reference writes / reference writes
Write F1	2PR / (P + R)
Escalation rate	trials with a transfer / trials; split into appropriate, unnecessary, missed
Blocked rate	guard-blocked proposed calls / proposed calls
Bootstrap CI	resample tasks with replacement, recompute mean success (or paired difference), take 2.5th and 97.5th percentiles

Decisions per tool call. allow, block (reason back to the model), approve (operator sign-off), escalate (hand to a human).

Two humans. the customer confirms intent for every account change; an operator signs off risk for irreversible actions. Timeout means deny.

Debugging loop. metrics, then failure taxonomy, then the trace timeline, then the first wrong step, then the root cause, then one fix, then a paired re-run.